VIETNAM ACADEMY OF SCIENCE AND TECHNOLOGY
UNIVERSITY OF SCIENCE AND TECHNOLOGY OF HANOI

BASIC PROGRAMMING

Lecture 4: Functions

Dr. NGUYEN Hoang Ha
Dr. NGUYEN Minh Huong
Dr. LE Huu Ton

Possible to write thousands lines of code with mixed structures?

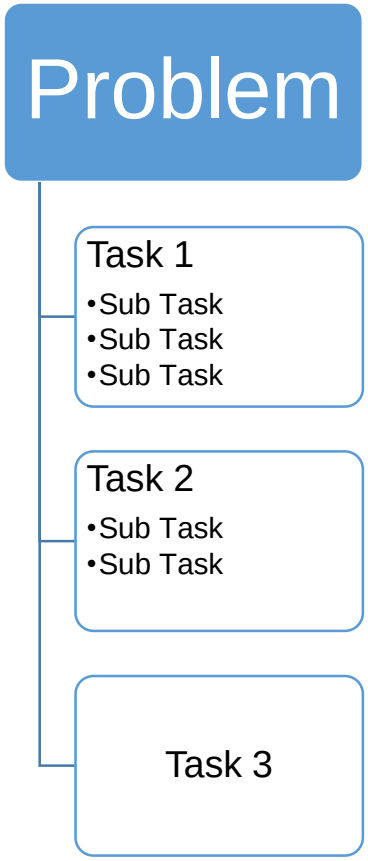
How to organize a big program?

Agenda

- Top-down design
- Defining functions
- Calling functions
- Recursive functions
- The Make utility
- Practical time

Top-down Design

- Large problems:
  - Highly complex
  - Heavy computation
  - Hard to implement
- Break the problem down into tasks and tasks down into subtasks
  - Less complex
  - Lighter computation
  - Easy to implement
- Eventually Sub Tasks become manageable and implementable in C



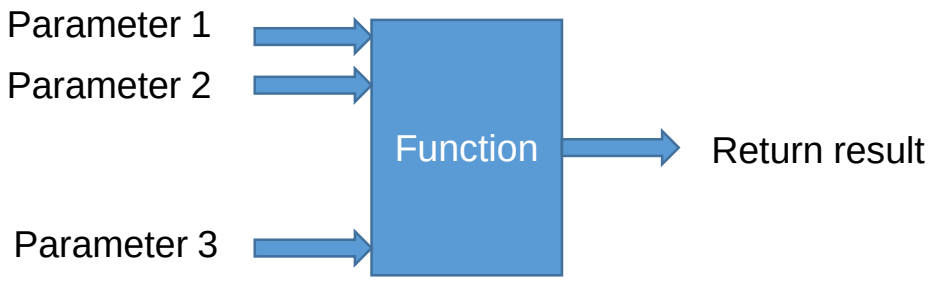
Top-down Design in Programming

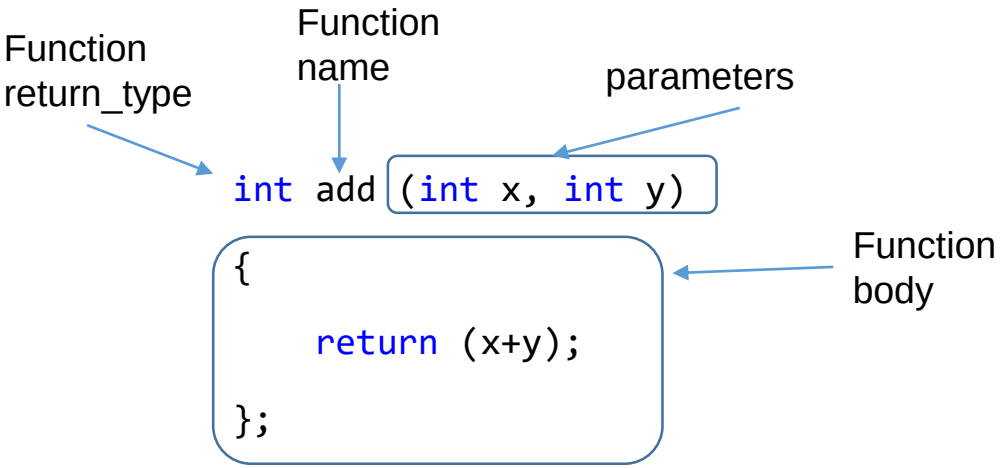
- Function: a group of statements performing specific tasks
  - Two or more commands
  - Has a name to be reused
- Goal: divide and conquer

Advantages of Functions

- Keep the main program smaller → easier to read and modify
- Enhance
  - Code management
  - Reusability
  - Readability: make main program smaller, focus on big points
- Reduce
  - Repetition code
  - Risks of error/bug
- Ease: writing, reading, debugging the code

Structure of Functions





- Number of arguments: zero to any
- Return type: void or any datatype

Types of Functions in C

- Build-in functions:
  - Found in standard libraries
  - Eg:
    - `printf(), scanf()` in `<stdio.h>`
    - `sqrt(), pow()` in `<math.h>`
- User-defined functions

```
#include <stdio.h>
void functionName()
{
    ... .. ...
}
int main()
{

    ... .. ...

    functionName();

    ... .. ...
}
```

Standard Libraries

- Common libraries

Header file	Descriptions
`stdio.h`	Input/Output functions
`conio.h`	Console Input/Output functions
`stdlib.h`	General utility functions
`math.h`	Mathematics functions
`string.h`	String functions
`ctype.h`	Character handling functions
`time.h`	Date and time functions
`float.h`	Limits of float types
`limits.h`	Size of basic types
`wctype.h`	Functions to determine the type contained in wide character data.

- All of standard libraries: https://www.cplusplus.com/reference/clibrary/

Defining Functions

Function Declaration

- Function must be declared before invoked
- Syntax:

```
return_type function_name (parameter_list){
    body of the function
}
```

Function Declaration (cont')

- Choose a meaningful function name
  - Convention: should be a verb, using _ to separate words
- Define a contract
  - Inputs
    - Arguments – choose type and name, none → void
    - Will the function change the parameter's value?
  - Output
    - Only one ; by convention, 0 means good
    - Only one: datatype, void means nothing.
- Body: set of statements (in different structures)
- Function cannot be defined in another function

E.g. of a Simple Function

```c
#include <stdio.h>

//The definition of a very simple function returning nothing - void
void say_hello()
{
    printf("Hello from a function\n");
}

//Every program in C must have 01 main() function as the entry
int main()
{
     /*calling function*/
     say_hello();
     return 0;
}
```

E.g. of another Simple Function

```c
#include <stdio.h>
int addition (int num1, int num2)
{
    int sum;
    sum = num1+num2;
    return sum;
}

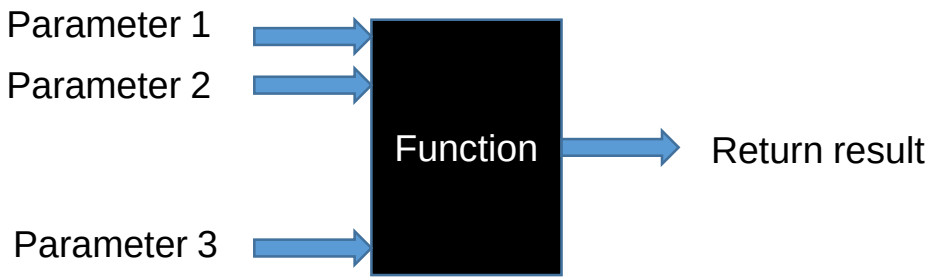
int main ()
{
     int var1, var2;
     printf("Enter number 1: ");
     scanf("%d",&var1);
     printf("Enter number 2: ");
     scanf("%d",&var2);

     /* Calling the function here */
     int res = addition(var1, var2);
     printf ("Output: %d", res);

     return 0;
}
```

Function prototype

- Think of function as a backbox → as a function consumer, sometimes programmers do not need to look at function body



- Declare prototype (header) of function:

```
return_type function_name (parameter_list);
```

Declare Function with Prototype

```c
#include <stdio.h>

//Firstly declare function prototype
void say_hello ();

int main()
{
     //calling function, even when not yet define it
     say_hello ();
     return 0;
}

//Declare full function after invoking it at the main()
void say_hello ()
{
    printf ("Hello from a function\n");
}
```

Function Declared in Separate Files

- A big program shoud not be written in a single file
  - Follow “divide and conquer” mindset
- Request to use existing code from outer file by using C processing directive `#include`

E.g. of Function in a Separate File

File say_hello.h

```c
#include <stdio.h>
//Declare function in a separate file
void say_hello()
{
    printf("Hello from a function\n");
}
```

File main.c

```c
#include "say_hello.h"
int main()
{
     //calling function declared in say_hello.h
     say_hello();
     return 0;
}
```

Header File (*.h) and Implementation File (*.c)

- Function creators want to hide code body
  - Blackbox mindset: separate template and implementation code
  - Security purposes → Do not ship the body code, but only function prototype (in *.h files) and binary code (in *.dll and/or *.lib)
- A library is often written in two files
  - *.h for prototypes
  - *.c for body of functions → compiled into binary files (*.dll or *.lib)

E.g. of Function in 02 Files

File say_hello.h

```
#ifndef SAY_HELLO_H
#define SAY_HELLO_H
//Function prototype
void say_hello();
#endif
```

File say_hello.c

```
#include <stdio.h>
//implementation of the function
void say_hello()
{
    printf("Hello from a function\n");
}
```

File main.c

```
#include ”say_hello.h”
//calling the function
int main()
{
    say_hello();
}
```

Compile with gcc

```
gcc .\say_hello.c .\main.c -o test_function
```

Calling Functions

Function Using

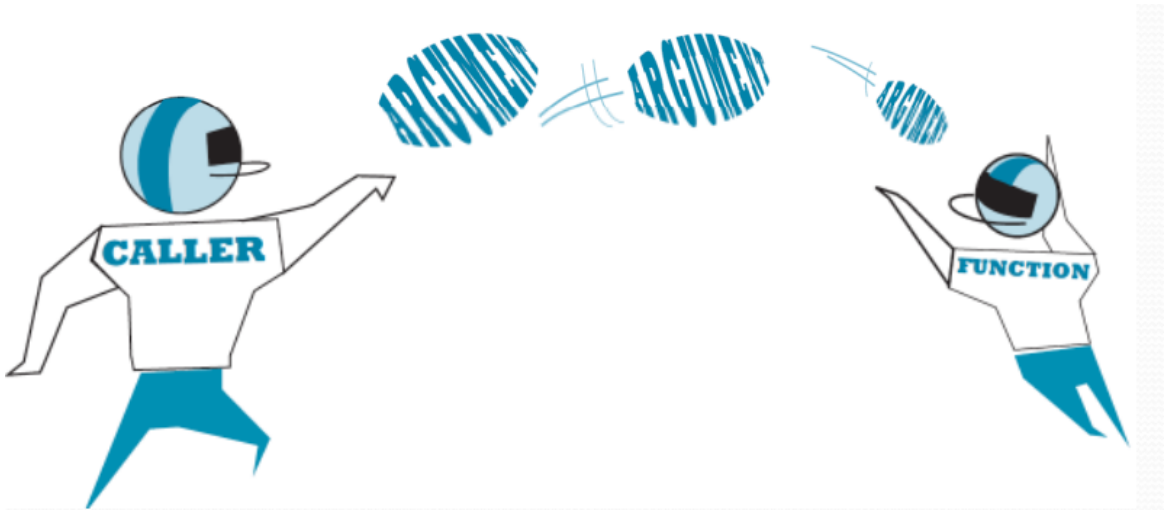
- before calling functions
  - Use `#include <header_file>` for standard libs
  - Use `#include "header_file"` for user-defined libs

- Call (invoke) by:
  - Write function name
  - Provide parameter values if required:
- E.g.:
  - `printf("Hello world");`
  - `double result = pow(2,3);//pow(2,e) is called function call`

Calling a Function

- Calling a function means running the code in its body
- Function is defined → memory is occupied. But if it is not called, the code never runs.
- To call:
  - Use its name
  - Followed by () with values for parameters if any.
  - C programming has two parameter passing schemes:
    - call by value
    - call by pointer (studied later in session 6).

Call by Value



- The caller pass the value to the function
  - If the parameter (argument) is a number → trivial
  - If the parameter is a variable → the clone of the variable is passed to the function → function can not modify the original variable

Call by Value Demo

```c
#include <stdio.h>

void double_num(int num){
    num *=2;
    printf("Inside function, the num is %d\n", num);
}

int main()
{
    int my_number = 10;
    double_num(my_number);

    printf("After calling a_number, my_number is still %d\n", my_number);
    return 0;
}
```

```
PS C:\TEACHING\BP\Lecture5> .\pass_by_value_demo.exe
Inside function, the num is 20
After calling a_number, my_number is still 10
```

Using const in Function Prototype

- Using const keyword for each parameter → add a constraint to the body: impossible to change the value of parameter (even the copied version)
  - Like constant variable declaration

```
void say_hello (const int times)
{
    for(int i=0;i<=times;i++)
        printf("Hello from a function\n");
}
```

  - Will discuss on const when study call by pointer

Recursive Functions

Recursive Function

- Recursive is the process of repeating in a self-similar way
- Recursive function in C: the body consists of statement invoking the name of function itself.
- When writing a recursive function, must define an exit condition to avoid infinite loop

Factorial Function in Recursive Format

```c
#include<stdio.h>
int find_factorial(int n)
{
   //Factorial of 0 is 1
   if(n==0)
      return(1);

   //Function calling itself: recursion
   return(n*find_factorial(n-1));
}
int main()
{
   int num, fact;
   printf("\nEnter any integer number:");
   scanf("%d",&num);

   //Calling our user defined function
   fact =find_factorial(num);

   //Displaying factorial of input number
   printf("\nfactorial of %d is: %d",num, fact);
   return 0;
}
```

Factorial Function in Non-Recursive Format

```c
#include<stdio.h>
int find_factorial(int num)
{
    if(num<=0)
        printf("\n Invalid Number! ");
    else{
        int fact=num;
        int i=1;
        while(i<num){
            fact=fact*i;
            i++;
        }
        printf("\n Factorial = %ld",fact);
    }
}
int main()
{
   int num, fact;
   printf("\nEnter any integer number:");
   scanf("%d",&num);

   //Calling our user defined function
   fact =find_factorial(num);

   //Displaying factorial of input number
   printf("\nfactorial of %d is: %d",num, fact);
   return 0;
}
```

Recursive Function Pros vs. Cons

Pros

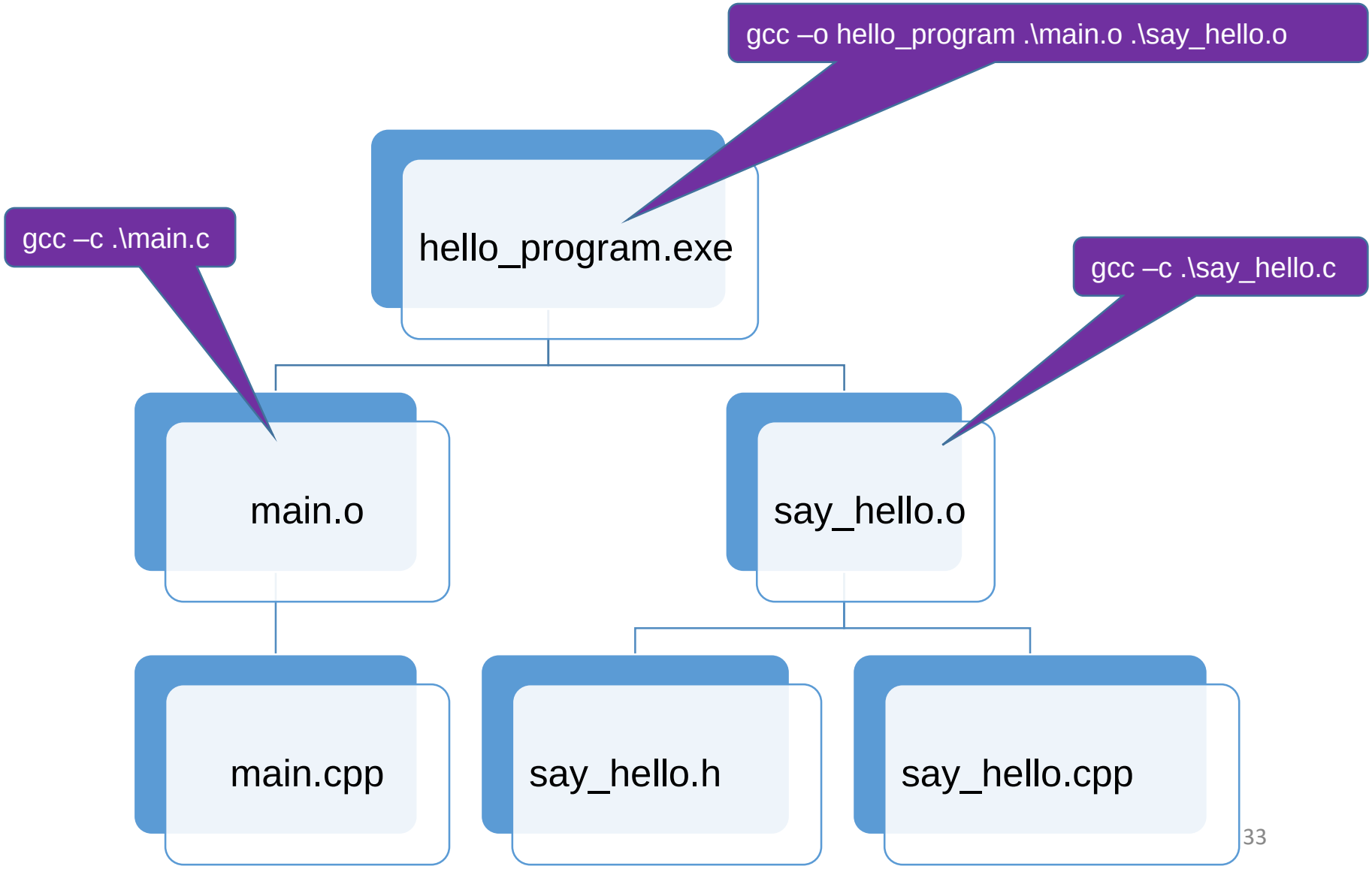
- Simpler to write
- Easier to debug
- Code is short

Cons

- Consume more memory
- Risk to infinite loop

The Make Utility

Context - A Project Structure



Motivation

- Small programs → all in one single cpp file
- Bigger programs → modularize into functions, files
  - Many lines of code
  - Multiple programmers
  - Multiple components
  - Manual building is laborious
- How to automate the building process?

A Make file

- "makefile" is put at the folder of source files

```
hello_program: say_hello.o main_say_hello.c
    gcc -o hello_program say_hello.o main_say_hello.c

say_hello.o: say_hello.c
    gcc -c say_hello.c
```

- To compile the whole project ,just type "make"

Makefile rule

- Each section consists of 2 lines
  - Outputfile: inputfile1, inputfile2 …
  - Compiling command
- Example:

```
hello_program: say_hello.o main_say_hello.c
    gcc -o hello_program say_hello.o main_say_hello.c

say_hello.o: say_hello.c
    gcc -c say_hello.c
```

Practical time

Practical time

1. Write a function to compute average of two numbers
2. Write a program calling above function to compute average of 4 numbers.